GRIDLY: A SEARCH SYSTEM OVER FORMULA ONE HISTORICAL ARCHIVES

Emilie Kernivinen
up202306154@up.pt
Faculty of Engineering - University of Porto
Porto, Portugal

João Marques
up202307612@up.pt
Faculty of Engineering - University of Porto
Porto, Portugal

João Ferreira
up202305204@up.pt
Faculty of Engineering - University of Porto
Porto, Portugal

José Ferreira
up202305478@up.pt
Faculty of Engineering - University of Porto
Porto, Portugal

ABSTRACT

Formula One history is scattered across sources that are rarely used together: results and standings are kept as structured data, while the reasons behind them are told in free text. We build a search system that unifies both, combining Jolpica F1 records with English Wikipedia articles, linked through the article URLs stored by Jolpica. A reproducible pipeline collects, cleans and links the sources into 2058 documents about races, drivers, teams and circuits from 1950 to 2026, which are indexed in Apache Solr with a separate analysis for free text and for names. We evaluate three ranking setups on thirteen information needs. Our schema alone raises the mean average precision from 0.25, with Solr's default field types, to 0.34, and weighting the sections of each document and filtering by document type raises it to 0.54, but needs that depend on a cause and its effect, such as a circuit redesigned after an accident, remain hard for a search based only on words.

CCS Concepts

• **Information systems** → Information retrieval query processing.

Keywords

Formula One, Motorsport, Information Retrieval, Search System, Wikipedia, Jolpica F1, Data Preparation, Data Quality, Conceptual Model, Collection Characterization, Apache Solr, BM25, Evaluation

ACM Reference Format:
Emilie Kernivinen, João Marques, João Ferreira, and José Ferreira. 2026. GRIDLY: A SEARCH SYSTEM OVER FORMULA ONE HISTORICAL ARCHIVES. In *Proceedings of PRI (G67).* ACM, New York, NY, USA, 12 pages.

1 INTRODUCTION

Formula One is the highest class of single-seater motorsport, and its World Championship has been held every year since the first race in 1950 [13]. In more than seven decades, the sport has accumulated a very large historical record, with over a thousand races and hundreds of drivers, teams and circuits that have come and gone [4]. Interest in the sport has also grown in recent years: Formula 1 reports a global fanbase of 827 million in 2025, 63% more than in 2018 [1], bringing many new followers who want to learn about its past but have no easy way to explore it.

This history is available in two forms, which are rarely found together. Structured data, such as race results, retirement reasons and championship standings, is maintained by open projects like Jolpica F1 [4], the successor to the Ergast API. Free text, such as race reports, driver biographies and team histories, is found mainly in collaborative encyclopedias like Wikipedia [14]. Each form answers different questions: the results show what happened, while the text explains why. Finding races decided by a late mechanical failure, for example, requires matching the retirement reason in the results with the narrative of the race. Answering this kind of question is the purpose of an information retrieval system [6], and it makes Formula One a good domain for one close to a real-world scenario.

In this project, we present Gridly, a search system that integrates both types of information. We build a collection in which each race, driver, team and circuit is one document that joins its structured fields from Jolpica F1 with its text from Wikipedia, index it in Apache Solr [11], and evaluate three retrieval setups on questions that need the text, such as finding the races decided by a late mechanical failure.

The remainder of this paper is organized as follows. Section 2 describes the dataset, including its sources, preparation, quality, structure and characterization. Section 3 presents the search scenarios used to evaluate the system. Section 4 explains how the documents are indexed and searched and the three setups we compare, and Section 5 evaluates them. Finally, Section 6 concludes and outlines future work.

2 DATASET

2.1 Data Origin and Description

Two complementary sources were selected, which, when combined, cover the full history of the sport: the Jolpica F1 database and English Wikipedia.

Table 1 characterizes both sources in their raw form. For Jolpica, rows and columns are summed over its 11 tables; for Wikipedia, each row corresponds to an article, and each column to a field stored for it.

Source	Format	Files	Rows	Columns	Size (MB)
Jolpica F1	CSV	11	141029	110	7.30
Wikipedia	JSON	2225	2225	7	77.37

Table 1: Initial data sources characterization [4, 14]

Jolpica F1 [4] is an open-source project that succeeds the Ergast API, which was discontinued at the end of 2024. Its relational database holds the results of every World Championship race since the inaugural 1950 season, together with additional information such as starting grids, finishing positions, retirement reasons, team lineups, and championship standings. We used its CSV dump instead of the REST API, which would require around 260 paginated requests under an hourly rate limit. Wikipedia [14] provides the text: the narrative of each race, the biography of each driver and the history of each team and circuit. The articles were obtained in wikitext through the MediaWiki Action API [8], returning several articles per request.

The two sources are linked directly through Jolpica, which stores the Wikipedia URL of each race, driver, team and circuit. This avoids matching by name, which would be error-prone for drivers with the same name or races whose title changed over the years. Both sources are released under open licenses (CC BY-NC-SA 4.0 for Jolpica and CC BY-SA 4.0 for Wikipedia) that allows their use in an academic project.

2.2 Data Preparation

The pipeline (Figure 1) transforms the two raw sources into a validated collection of documents. It is implemented in Python as five scripts, each reading the output of the previous one. Raw downloads, intermediate files and the final collection are kept in separate folders.

Data Ingestion downloads the Jolpica dump, keeps the 11 tables needed and records the version used (12 September 2026), since the dump link always points to the latest one. *Entity Structuring* then joins these tables with Pandas [7] into one record per race, driver, team and circuit, with values such as the winner and retirements of a race or the career totals of a driver. Jolpica has one team row per chassis–engine combination ("Cooper-Climax", "Cooper-Maserati"), so rows that point to the same article were merged into one team, and only races already held were kept.

Each record carries the link to its Wikipedia article, which the *Wikipedia Crawler* uses to download the text through the MediaWiki API. Since Wikipedia blocks anonymous clients that send one request per page (HTTP 429), articles are requested in batches of 20, reducing about 2230 requests to 115. Each article is cached locally with its page and revision identifiers, which identify the exact version used and allow later runs to download only what is missing.

The *Wikitext Parser* converts the wikitext to clean text with mwparserfromhell [5], removing tables, references, infoboxes and images and keeping the visible text of links. In *Schema Mapping & Filtering*, each article is split by its sections, which are mapped by keywords to a fixed set of text fields per document type (Table 4), and sections without prose, such as classification tables, are discarded. Documents with fewer than 100 words of text are rejected, with the reason recorded. The *Relational Linker* links the accepted ones to each other in both directions (races, drivers and teams, and races and circuits), keeping only links between documents that exist in the collection, and exports them in JSON and CSV. Finally, *Collection Characterization* computes the statistics and plots of Section 2.6, using Matplotlib [3], NumPy [2] and scikit-learn [9].

The whole pipeline runs with a single `make` command, which creates the virtual environment, installs the dependencies and runs the five steps in order. Each step also has its own target (`collect`, `structure`, `wikipedia`, `documents` and `analyze`), while `make clean` removes the generated files but keeps the downloads, and `make clean-all` removes everything.

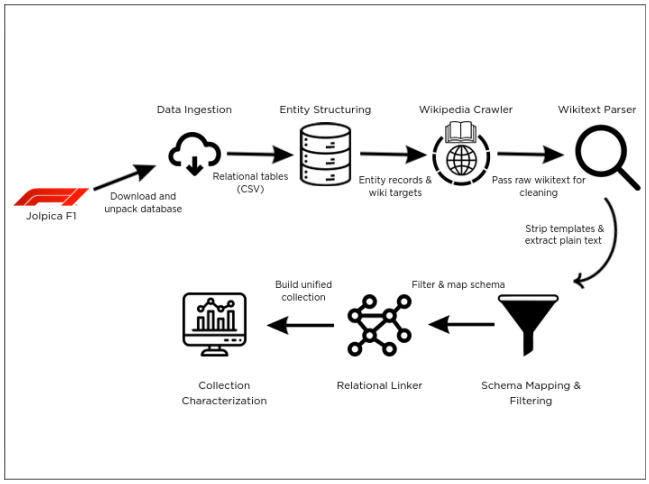


Figure 1: Data preparation pipeline

2.3 Data Quality

Table 2 lists the problems found and how they were handled. Most come from the links between the sources. In 28 cases, the Jolpica link leads to a **disambiguation page** ("Tony Brooks may refer to…"), usually because a more famous person has the same name. So that the fix survives a new run of the pipeline, they are resolved automatically by choosing the entry that mentions "Formula One" or "racing driver" (e.g., "Tony Brooks (racing driver)"), and each correction is recorded with the original link. The Long Beach circuit links to the article about the city, which is a valid article and cannot be detected automatically, so it was corrected by hand.

In the results, the **status** is a numeric code with no documentation. By grouping the values of the `detail` column ("Engine", "Spun off", "+1 Lap") under each code, we inferred its meaning: finished, finished with laps behind, retired by accident, retired by failure, disqualified and did not start. This made it possible to keep the retirement reasons, one of the most useful structured fields.

Most of the 164 **short documents** are drivers from the 1950s with one or two races, whose article is a single sentence followed by a table. The threshold was chosen by inspecting these articles: raising it to 150 words would reject 111 more documents, many of them with useful text.

2.4 Final Dataset

The final collection comprises 2058 documents of four types and 59 distinct fields, stored in a 22.1 MB JSON file (Table 3). Each document combines structured fields from Jolpica, used for filters, with text fields from Wikipedia, which are the main ones searched (Table 4). All documents also keep the article URL, the revision

Issue	Count	Handling
Links to disambiguation pages	28	resolved automatically
Link to the wrong article (Long Beach)	1	fixed manually
Links to missing articles	3	rejected
Documents under 100 words	164	rejected
Team rows sharing one article	40	merged
Undocumented numeric `status`	–	inferred from `detail`
HTML entities in names	1	decoded

Table 2: Data quality issues and how they were handled

identifier and their word count. An abridged example of a race document is shown in Appendix A.

	Races	Drivers	Teams	Circuits	Total
Jolpica records	1162	818	168	77	2225
Accepted	1116	719	147	76	2058
Rejected	46	99	21	1	167
Fields	34	27	24	25	59
Median words	827	690	1082	1238	–

Table 3: Final collection. The total of fields counts distinct fields, since 9 are shared by all types

Type	Structured fields	Text fields
Race	season, date, circuit, country, winner, pole, podium, fastest lap, starters, finishers, retirements	summary, background, qualifying, race, post_race
Driver	nationality, birth date, seasons, teams, wins, podiums, poles, championships	summary, biography
Team	names, nationality, seasons, wins, podiums, championships	summary, history
Circuit	locality, country, coordinates, Grands Prix, most wins	summary, history, layout, events

Table 4: Main fields of each document type

2.5 Conceptual Model

The conceptual model (Figure 15) describes the documents produced by the pipeline. An abstract **Document** holds the fields shared by all types (identifier, title, word count, Wikipedia identifiers and the `summary`). It is specialized into four classes, **RaceDocument**, **DriverDocument**, **TeamDocument** and **CircuitDocument**, each with its own structured attributes and text sections. Identifiers are canonical and readable, such as `race_2021_22` or `driver_hamilton`, and only documents with at least 100 words are accepted into the collection.

Each race has several drivers and at least two teams, and each driver participates in one or more races and drives for one or more teams. A team has at least one driver, not necessarily two: 47 Jolpica teams had a single driver in their whole history, mostly private entries of the 1950s. A team may have no linked race, which happens for three teams whose races were all rejected. Each race is held at exactly one circuit, which hosts one or more races; in the documents this link is optional, because the Pedralbes circuit was rejected and its two races have no circuit document. All relations are stored as lists of identifiers in both directions (e.g., `driver_ids` in a race and `race_ids` in a driver), allowing the search system to move from a document to the related ones.

2.6 Data Characterization

Document length is very uneven (Figure 7). In every type the mean is well above the median, pulled up by a few long articles: the longest driver document (Kimi Räikkönen) has 14454 words, twenty times the median. For drivers, length grows with the career: the number of race entries and the number of words have a Spearman correlation of 0.69 (Figure 12). For races, length depends on the period (Figure 2): the median goes from about 330 words in the 1960s to 1849 in the 2000s, almost six times more. The 2000s are the peak, which suggests that these seasons were written about as they happened by a large community of editors, while older races were documented later and with fewer sources. This matters for the search system: a long document has more chances of matching a query, so a ranking that does not normalize length well will tend to favor recent races.

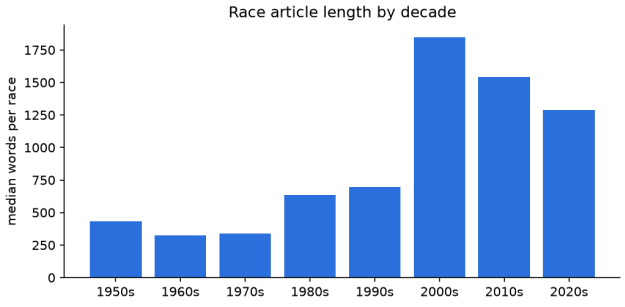


Figure 2: Median words per race document, by decade

The collection has 2.79 million tokens and a vocabulary of 34348 terms, 37% of which appear only once. Stopwords make up 45% of all tokens. Term frequency follows Zipf's law [6] between ranks 10 and about 2000 (Figure 10), with the usual deviations at both ends: the most frequent words are less frequent than the law predicts, and the tail falls faster. The vocabulary grows sublinearly, as Heaps' law predicts (Figure 11). As documents are read grouped by type, the growth almost stops during the races, which reuse the names and terms of the biographies, and rises again with the teams.

After removing stopwords, "championship", "prix", "race" and "formula" appear in about 95% of the documents, so they will have little weight in a ranking based on inverse document frequency. The terms specific to each type are more telling (Figure 14): races use "lap", "pit", "qualifying" and "tyres", teams "engine", and circuits "track" and "turn".

To see how the language of the race reports changed over time, we extracted the keywords of each decade with TF-IDF [6], using

scikit-learn and removing the names of drivers, teams and circuits, which would otherwise dominate. Table 5 shows that the keywords follow the history of the sport: privateer and works entries in the 1950s, turbo engines in the 1980s, refuelling stops and Friday sessions in the 2000s, and tyre compounds, penalties and the safety car in the 2010s. Some noise remains, such as "utc", from the start times given in recent articles.

Decade	Keywords
1950s	manuel, british, entered, privateer, argentine
1960s	manufacturers, cup, british, works, entered
1970s	cup, manufacturers, austrian, south, swedish
1980s	tag, turbo, brazilian, frenchman, italian
1990s	pre, report, failed, spun, failure
2000s	friday, constructors, stops, sessions, saturday
2010s	turn, penalty, tyre, compound, safety
2020s	utc, local, tyre, event, topped

Table 5: Top TF-IDF keywords of race documents by decade

In the structured fields, engine failures are the most common retirement reason (1889 cases), but accidents, collisions and spins together exceed 2500 (Figure 3). British (142) and American (131) drivers are the most common (Figure 8), the latter because the Indianapolis 500 counted for the championship until 1960. Finally, every race has a summary and 85% have a race report, but only 42% describe qualifying and 27% the aftermath (Figure 9), so some queries can only be answered for part of the collection, mostly recent seasons.

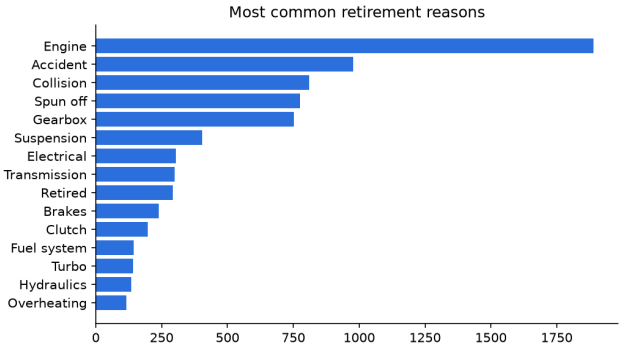


Figure 3: Most common retirement reasons

3 SEARCH SCENARIOS

Table 6 lists the information needs used to evaluate the system. They were chosen so that they depend on the text and not only on the structured fields, cover all four document types, can be expressed with varied vocabulary, and have answers in the collection, which we confirmed by finding relevant documents for each. For each need we defined what makes a document relevant and what does not. In N2, for example, a race is relevant only if it was affected by rain **and** had an unusual result, such as the 1996 Monaco Grand Prix, where only three cars finished. A wet race won by the favorite is not relevant.

ID	Information need	Type
N1	Races decided by a pit-stop or tyre strategy	race
N2	Wet races with a surprising result	race
N3	Safety car periods that changed the result	race
N4	Collisions between teammates	race
N5	Race weekends with a fatal accident	race
N6	Disqualifications for technical infringements	race
N7	Drivers who returned after a serious injury	driver
N8	Drivers who succeeded at Le Mans or Indianapolis	driver
N9	Teams that collapsed for financial reasons	team
N10	Technical innovations that were later banned	team
N11	Teams that were renamed or taken over by another company	team
N12	Circuits whose layout was changed after a serious accident	circuit
N13	Circuits that drivers boycotted or considered too dangerous	circuit

Table 6: Information needs

We deliberately included some needs that are hard to satisfy. In N1, the same idea appears as "undercut", "two-stop strategy" or "gambled on not needing a pit stop"; in N4, the text often gives only the two names, without saying that they are teammates; N7 has few documents that use the words of the query, even though there are well-known cases such as Niki Lauda or Felipe Massa; and in N10, "banned" mostly returns the ban on turbo engines, which affected every team and is not relevant. These cases expose the limits of a search based only on words.

4 INFORMATION RETRIEVAL

To search the collection we use Apache Solr [11], an open-source search platform. Solr receives the documents, organizes their content in an index that makes search fast, and answers each query with a list of documents ordered by relevance. This order is computed with BM25 [10], Solr's default ranking function. BM25 gives more weight to words that are rare in the collection and compensates for document length, which matters here because the length of our documents is very uneven (Section 2.6). Solr runs in a Docker container, and a single script creates the container, defines the fields and loads the documents, together with a copy without our schema used for comparison (Section 4.3), so the whole index can be rebuilt at any time.

4.1 What Is Indexed

Each race, driver, team and circuit is a separate document, so every search result is one complete entity that a user can read on its own. Relations between documents are kept as lists of identifiers, as described in Section 2.5.

Not all fields should be searched in the same way. A race report is free text, a driver's name must not be altered, and a season is a number. We therefore grouped the fields of the schema into the five kinds shown in Table 7, each with its own Solr field type.

Narrative text uses a type we defined, `f1_text`, with the full analysis described in Section 4.2. Names use a second type of our own, `f1_name`, which only ignores case and accents. Categories and identifiers, such as the document type or the list of driver identifiers of a race, are stored as `string` and only match their exact value. Numbers and dates use Solr's numeric and date range types, so they can be searched by range, such as the seasons from 1980 to 1989. All fields are indexed except the Wikipedia URL, which is only displayed. The title and the ten text sections are also copied into a single field, `all_text`, which is used by the baseline (Section 4.3).

Kind	Solr type	#	Example fields
Narrative text	`f1_text`	11	summary, race, biography, history, `all_text`
Names	`f1_name`	14	title, winner, drivers, teams, circuit
Categories	`string`	11	type, country, nationality, driver_ids
Numbers	`pint`, `pdouble`, `plong`	21	season, wins, starters, latitude
Dates	`DateRange`	2	date, date_of_birth

Table 7: Fields of the Solr schema, by kind, without the document identifier

4.2 Text Analysis

Before being indexed, text goes through an analyzer: a tokenizer splits it into words, and a sequence of filters then transforms those words. Narrative text goes through the following steps:

- **Tokenization** splits the text at spaces and punctuation.
- **Possessive removal** turns "Senna's" into "Senna".
- **Accent folding** adds a version without accents, so that "Raikkonen" also finds "Räikkönen".
- **Lowercasing** makes searches independent of case.
- **Stop word removal** discards the 35 words of Solr's English list, such as "the" or "in", which carry little meaning and make up 29% of the tokens of the text fields. This list is much shorter than the one used in Section 2.6, so words such as "his" or "after" are kept.
- **Stemming** reduces words to a common root with the Porter algorithm, so that "crashed" and "crashing" match each other.

For example, the sentence *"Räikkönen's crashing in the rain"* is reduced to the terms *raikkonen*, *räikkönen*, *crash* and *rain*. The same steps are applied to the queries, so that both sides end up in the same form. Names only go through tokenization, accent folding and lowercasing, because removing word endings could change a name and stop words can be part of one.

We did not add a list of synonyms. Words with the same meaning, such as "banned", "outlawed" and "prohibited", are included by hand in the query of each need, and handling them automatically is left as future work.

4.3 Retrieval Setups

We compare three setups. All use eDisMax [12], the Solr query parser designed for free text, and all run the same query for each information need. A query usually lists the main words of the need and their common variants: for N1 it contains *undercut*, *"two-stop"* and *gamble*, among others. In N3 and N4 single words were not enough, since almost every race report mentions the safety car, teammates and collisions, so their queries are made of short phrases instead, such as *"collided with his teammate"*. A number after a phrase (`~4`) allows that many extra words between its words, so that *"collided with his McLaren team-mate"* also matches. A document does not need to contain all of the words or phrases to be returned, and documents are always ordered by relevance alone. Table 8 describes the parameters that were used.

Name	Meaning	Schemaless	Baseline	Boosted
`q`	Words to search for	same	same	same
`qf`	Fields to search and their weights	11 fields, equal	one field	several (Table 9)
`pf2`	Bonus when pairs of query words appear close together	–	–	several (Table 9)
`ps2`	Maximum distance between words for that bonus	–	–	3 positions
`fq`	Restricts the results without changing their score	–	–	type of the need

Table 8: Query parameters of each setup

The **schemaless** setup shows what Solr does without our schema: the same documents are loaded into a second index with Solr's default configuration, which guesses each field type from its first value. Text fields are only split and lowercased, without stop words, stemming or accent folding, and as there is no `all_text`, the query searches the title and the ten text sections with equal weight, over the whole collection.

The **baseline** uses our schema with a minimal query. The title and all Wikipedia sections are copied into a single field, so a word counts the same wherever it appears, and the search covers the whole collection.

The **boosted** setup uses what we know about the structure of the documents (Table 9). The summary has the highest weight, since it concentrates the main facts of an article, and the sections that narrate what happened count more than background information. Documents receive a bonus when consecutive words of the query appear close together, such as *serious* and *injury* in N7. We apply this bonus to pairs of words because most of our queries are lists of keywords: a bonus that required all of them to appear together had no effect on any need. Results are restricted to the document type requested by each need. The lists of teams and drivers are also searched, with a low weight, so that a query with the name of a driver or a team also finds the races in which they took part. The weights were chosen manually while testing queries, and the same configuration is used for every need.

4.4 Demonstration

Once the script mentioned above has run, all 2058 documents can be searched. Queries can be tested in the Solr interface, which shows

Field	Word weight	Word pair weight
Summary	3	4
Race report, biography, history	2	3
Qualifying, post-race	2	2
Title	2	–
Other text sections, teams, drivers	1	–

Table 9: Field weights in the boosted setup

every parameter and how each score was computed, or in a small web application that we built (Figure 4). There, a user types a query, restricts it by document type and range of seasons, and chooses one of the three setups or shows them side by side. Each result shows its main structured fields and an excerpt of the text with the query words highlighted, and the information needs of Table 6 can be run with a single click.



Figure 4: Search page of the web application

As an example, for N3 (safety car periods that changed the result) we search for the phrase *"safety car"* together with phrases such as *"lost the lead"*, *"wiped out"* and *"pitted under"* (Appendix B). The baseline finds 1364 documents, the boosted setup 967, since it only considers races, and the schemaless setup 759, since without stemming fewer forms of each word match. Only three documents appear in all three top 10 lists, and Table 10 shows the first three positions; the third result of the baseline is not even a race, but the biography of Nicholas Latifi, whose crash caused the safety car of the 2021 Abu Dhabi Grand Prix. The configuration alone therefore changes what the user sees on the first page. Which list is better can only be decided by judging the results.

Rank	Schemaless	Baseline	Boosted
1	2026 Belgian GP	2014 Hungarian GP	2026 Belgian GP
2	2003 British GP	1998 British GP	2000 German GP
3	2021 Abu Dhabi GP	Latifi (driver)	2003 British GP

Table 10: Top results for N3 in each setup

5 EVALUATION

We evaluate the three setups on the thirteen information needs of Table 6. For each need, we wrote down beforehand what makes a document relevant and what does not, so that every judgement follows the same rule. Appendix B lists these criteria together with the query and filter of each need, and Section 5.1 describes one need of each document type in more detail. For example, a race is relevant to N1 only if the text says that the strategy decided who won; a race that only mentions routine pit stops, or where the strategy changed positions outside the fight for the victory, is not.

Judgements. Each setup returns its top 10 documents for every need. The three lists are merged without duplicates, which gives between 16 and 28 documents per need, and each document is judged by hand as relevant (1) or not relevant (0). To decide, we read an excerpt of the document in which the query words are highlighted and, when that is not enough, the full article. Recall is measured relative to this merged list, not to the whole collection. This is a limitation: a relevant document that no setup placed in its top 10 is never judged, so recall is overestimated, and a document found by only one setup counts as missed by the others. Judging the whole collection was not feasible, and since all setups share the same pool, the comparison remains fair.

Metrics. For each need and setup we compute precision at 10 (P@10), the share of relevant documents in the first page of results, and average precision (AP), which also rewards placing the relevant documents near the top. The mean of the AP values over all needs (MAP) gives a single number per setup. We also plot the interpolated precision at eleven recall levels, averaged over all needs (Figure 6).

	Schemaless		Baseline		Boosted	
Need	P@10	AP	P@10	AP	P@10	AP
N1	0.40	0.22	0.40	0.27	0.70	0.54
N2	0.10	0.05	0.00	0.00	0.40	0.60
N3	0.80	0.51	0.70	0.46	0.60	0.35
N4	0.50	0.41	0.60	0.47	0.70	0.69
N5	0.90	0.69	0.40	0.26	0.90	0.58
N6	0.80	0.56	0.80	0.54	0.90	0.66
N7	0.00	0.00	0.50	0.41	0.60	0.42
N8	0.00	0.00	0.70	0.57	0.60	0.33
N9	0.30	0.25	0.40	0.36	0.60	0.66
N10	0.00	0.00	0.20	0.08	0.50	0.57
N11	0.60	0.30	0.80	0.51	0.90	0.61
N12	0.30	0.18	0.60	0.41	0.50	0.34
N13	0.20	0.07	0.10	0.03	0.70	0.72
Mean	0.38	0.25	0.48	0.34	0.66	0.54

Table 11: Results of each setup

Reading the precision-recall curve. Figure 6 answers the question: *to find a given share of the relevant documents, how much of what the user reads is useful?* Recall, on the horizontal axis, is the share of the relevant documents of a need that have already appeared in the ranking; precision, on the vertical axis, is the share of the documents read so far that are relevant. If a need has four relevant documents ranked 1, 3 and 6, recall 0.5 is reached at position 3 with precision 0.67, and recall 1.0 is never reached, so its precision

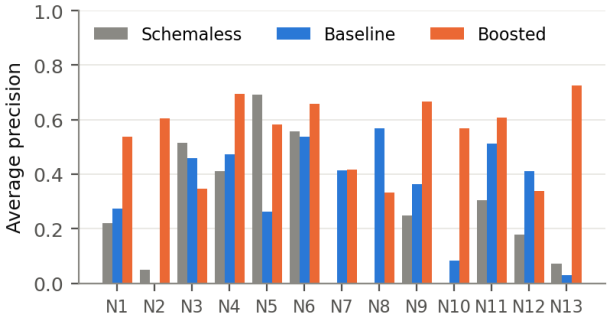


Figure 5: Average precision of each need in each setup

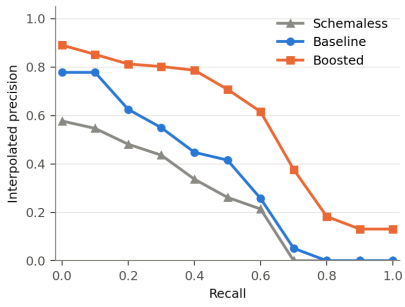


Figure 6: Interpolated precision-recall curve of each setup

is 0. Interpolation keeps, at each recall level, the best precision from that point onwards, so the curves never rise. We compute these values for each need and plot their average, one line per setup.

The boosted curve lies above the baseline at every recall level. The two start close (0.89 and 0.78), since both usually rank a relevant document first, and the gap grows in the middle of the curve: to find half of the relevant documents, 71% of the documents read are relevant in the boosted setup, against 42% in the baseline. At recall 0.8 and above the baseline drops to 0, because in no need does its top 10 contain even 80% of the relevant documents of the pool, while the boosted setup retrieves every relevant document in three needs and stays at 0.13. The schemaless curve is the lowest, starting at 0.58 and reaching 0 already at recall 0.7. Figure 13 in the appendix shows the same curve for each document type. The boosted setup is ahead for races, teams and circuits, but not for drivers, where the baseline has the higher precision up to recall 0.6, mostly because of N8.

Discussion. The boosted setup raises the MAP from 0.34 to 0.54 and the average P@10 from 0.48 to 0.66, and it has the higher AP in ten of the thirteen needs (Table 11 and Figure 5). The largest gains are in N13 (from 0.03 to 0.72), N2, N10 and N5, and a large part of them comes from the filter by document type. Of the 130 documents in the top 10 lists of the baseline, 31 (24%) are of the wrong type, and by our criteria these are never relevant: in N13, which asks for circuits, eight of the ten are race reports or biographies, such as the 1982 San Marino Grand Prix, which was *boycotted* by many teams for political reasons; in N5, five are biographies of drivers killed in accidents, such as François Cevert; and in N10, four are biographies, such as that of Jim Hall, whose banned "vacuum cleaner" car is described in his own article and not in that of a team. The field weights help in a different way. In N2 the baseline has no relevant document in its top 10, because a word counts the same wherever it appears: the 2023 Canadian Grand Prix is retrieved because rain fell during qualifying and a red flag came "only three minutes" after it began. The boosted setup gives more weight to the summary, where the result of the race is stated, and ranks four relevant races in its top 10.

The baseline is better in N3, N8 and N12, and these, together with N7, are also the needs with the lowest AP in the boosted setup. In all of them relevance depends on a relation that the words of the query do not express: a safety car that *changed* the result, a layout changed *because of* an accident, a return *after* an injury, or a *success* at Indianapolis rather than a mere participation. The boosted setup ranks first documents that contain the words but not the relation. In N3, its first result is the 2026 Belgian Grand Prix, where Kimi Antonelli lost the lead to a rival who benefited from a virtual safety car, but still won; in N8 it ranks the biographies of drivers who only took part in the Indianapolis 500, such as Carl Scarborough, who died during the 1953 race; and in N12 it returns circuits whose chicanes were changed to increase overtaking. Of the four needs we expected to be hard (Section 3), N7 is among the weakest, and neither Niki Lauda nor Felipe Massa, the examples given there, appears in the top 10 of any setup, while N1, N4 and N10 reached an AP between 0.54 and 0.69, thanks to queries with several variants of the same idea and, in N4, to phrases instead of single words.

Effect of the schema. Without our schema, the MAP falls to 0.25 and the average P@10 to 0.38. Part of the loss comes from the missing analysis (without stemming, *returned* no longer finds *return*), but most comes from the missing `all_text` field: with each section searched separately, BM25 measures how rare a word is in each field, and *injury* appears in 12% of the biographies but only 3% of the race sections, so a match in a race report counts more. Race reports fill the rankings, with 60 of the 130 results of the wrong type (31 in the baseline), and N7, N8 and N10 get no relevant document; for the same reason the schemaless setup beats the baseline in the race needs N2, N3, N5 and N6.

5.1 Detailed Information Needs

The four needs below, one of each document type, are described in more detail; the criteria of the other needs are in Appendix B. For each one we give the parameters sent to Solr by the boosted setup, apart from those used only to highlight the excerpts. These are the same for every need:

- `defType`: `edismax`; `fl`: `id, title, score`; `rows`: `10`
- `qf`: `title^2 summary^3 background qualifying^2 race^2 post_race^2 other biography^2 history^2 layout events teams drivers`
- `pf2`: `summary^4 qualifying^2 race^3 post_race^2 biography^3 history^3`
- `ps2`: `3`

The default operator (`OR`) and order (by score) of Solr are not changed, and each need below adds its own `q` and `fq`. The baseline

sends the same `q`, `fl` and `rows`, but with `qf=all_text`, without `pf2` and `ps2`, and without `fq`, so it searches the whole collection; the schemaless setup does the same on its index, with `qf` listing the title and text sections.

N1: Races decided by a pit-stop or tyre strategy

Relevance judgement: The goal is to find Formula One races whose winner was decided by the strategy of the teams: the number of pit stops, the lap on which a driver stopped, the choice of tyres, an undercut or overcut, or a gamble on stopping fewer times than the rivals. Since almost every modern race report describes pit stops, a race is not relevant when the stops were routine, or when the strategy only changed positions outside the fight for the victory. The 2019 Hungarian Grand Prix, won by Hamilton after switching to a two-stop strategy to pass Verstappen in the last laps, is relevant.

Search: The query combines the general terms (*pit stop*, *strategy*, *tyres*) with the specific forms in which the idea is written (*undercut*, *two-stop*, *one-stop*, *gamble*), since no single word covers every case. Results are restricted to races and ordered by score, so races in which several of these words appear in the summary or close together in the race report come first.

Query:

- q: `"pit stop" strategy undercut tyres "two-stop" "one-stop" gamble`
- fq: `type:race`

N7: Drivers who returned after a serious injury

Relevance judgement: The goal is to find Formula One drivers whose biography describes a serious injury that interrupted their career for some time **and** their later return to racing, as with Niki Lauda after his burns in 1976. Injuries that did not stop the career, injuries that ended it and fatal accidents are not relevant, even though their biographies use many of the same words.

Search: The query covers both halves of the need: the injury (*serious*, *injury*, *injured*, *hospital*, *burns*) and the return (*recovered*, *returned*, *comeback*). Results are restricted to drivers, and biographies where the two halves appear close together receive the word-pair bonus. The query cannot tell an injury followed by a return from one followed by retirement or death, which is why this need is hard.

Query:

- q: `serious injury injured recovered returned comeback hospital burns`
- fq: `type:driver`

N10: Technical innovations that were later banned

Relevance judgement: The goal is to find Formula One teams that created or pioneered a specific technical solution that the regulations later banned, such as a fan car or a six-wheeled car. A ban that applied to every team in the same way without being aimed at an innovation of that team, such as the end of turbo engines in 1989, is not relevant, and neither is an innovation that was never banned.

Search: The query joins the idea of a new solution (*innovation*) with the words for a ban (*banned*, *outlawed*, *prohibited*), and adds the names of the best-known banned solutions (*fan car*, *six-wheeled*, *ground effect*) as phrases. Results are restricted to teams. The word *banned* also appears in the history of many teams because of the turbo ban, so documents that only mention that general ban can still rank high.

Query:

- q: `innovation banned outlawed prohibited "fan car" "six-wheeled" "ground effect"`
- fq: `type:team`

N13: Circuits that drivers boycotted or considered too dangerous

Relevance judgement: The goal is to find Formula One circuits that drivers or teams boycotted or refused to race at for safety reasons, or that were described as too dangerous. Boycotts for political or contractual reasons are not relevant, and neither are circuits where accidents happened without any criticism of the safety of the track itself.

Search: The query includes the actions of the drivers (*boycott*, *boycotted*, *refused*, *protest*, *criticized*) and the judgement of the track (*dangerous*, *unsafe*, *safety concerns*). Results are restricted to circuits. Words such as *dangerous* and *drivers* also appear in plain descriptions of accidents, and *boycott* in political disputes, so some circuits match without being relevant.

Query:

- q: `boycott boycotted refused dangerous unsafe "safety concerns" protest criticized drivers`
- fq: `type:circuit`

6 CONCLUSION AND FUTURE WORK

We built a reproducible collection of 2058 Formula One documents that joins the results of Jolpica F1 with the text of Wikipedia, characterized it, and indexed it in Apache Solr with a different analysis for narrative text, names and categories. On thirteen information needs judged by hand, our schema raises the MAP from 0.25 to 0.34 over Solr's default field types, and a setup that weights the sections of each document and filters by type raises it to 0.54, and the average P@10 from 0.48 to 0.66, over a baseline that searches all the text as one field. Most of the gain comes from removing documents of the wrong type and from favoring the summary, and needs whose words appear in almost every race report required queries made of short phrases. The needs that remain hard are those in which relevance depends on a relation between two facts, such as an accident and the change of layout that followed it, which a search based only on words cannot see.

The most challenging part was turning Wikipedia into usable text: the API blocks one-request-per-page clients, the wikitext mixes prose with tables and templates, and some links led to disambiguation pages, which we solved with batched and cached requests, a prose-only parser and automatic link resolution.

As future work, we plan to add semantic search with text embeddings, combined with BM25 in a hybrid ranking, and compare it with the current setups on the harder information needs, where the same idea is written in different ways. We will also explore synonyms and query expansion for racing vocabulary, add filters such as nationality or team to the web application, and judge longer result lists, so that recall is estimated over a larger pool.

References

[1] Formula 1. 2025. Formula 1's Record-Breaking 2025 Season in Numbers. https://www.formula1.com/en/latest/article/formula-1s-record-breaking-2025-season-in-numbers.irq7aR8PcyAw7ysO72vJn. Accessed 2026-11-11.

[2] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. Array Programming with NumPy. *Nature* 585, 7825 (2020), 357–362. doi:10.1038/s41586-020-2649-2

[3] John D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95. doi:10.1109/MCSE.2007.55

[4] Jolpica F1 contributors. 2026. Jolpica F1: Open-Source Formula 1 Data (Successor to the Ergast API). https://github.com/jolpica/jolpica-f1. CSV database dump of 2026-09-12. Accessed 2026-09-27.

[5] Ben Kurtovic. 2025. mwparserfromhell: A Python Parser for MediaWiki Wikicode. https://github.com/earwig/mwparserfromhell. Version 0.7.2. Accessed 2026-09-27.

[6] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK.

[7] Wes McKinney. 2010. Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference*, Stéfan van der Walt and Jarrod Millman (Eds.). SciPy, Austin, Texas, USA, 56–61. doi:10.25080/Majora-92bf1922-00a

[8] MediaWiki contributors. 2026. MediaWiki Action API. https://www.mediawiki.org/wiki/API:Main_page. Accessed 2026-09-27.

[9] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[10] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.

[11] The Apache Software Foundation. 2026. Apache Solr. https://solr.apache.org. Version 10.1. Accessed 2026-11-11.

[12] The Apache Software Foundation. 2026. The Extended DisMax (eDisMax) Query Parser. Apache Solr Reference Guide. https://solr.apache.org/guide/solr/latest/query-guide/edismax-query-parser.html. Accessed 2026-11-11.

[13] Wikipedia contributors. 2026. History of Formula One. https://en.wikipedia.org/wiki/History_of_Formula_One. Accessed 2026-11-11.

[14] Wikipedia contributors. 2026. Wikipedia, The Free Encyclopedia (English). https://en.wikipedia.org. Accessed 2026-09-27.

A Example Document

Abridged race document (`race_2021_22`); long lists and texts are cut with "…".

```
{
 "id": "race_2021_22",
 "type": "race",
 "title": "2021 Abu Dhabi Grand Prix",
 "season": 2021, "round": 22,
 "date": "2021-12-12",
 "circuit": "Yas Marina Circuit",
 "circuit_id": "circuit_yas_marina",
 "country": "UAE",
 "winner": ["Max Verstappen"],
 "winning_team": ["Red Bull"],
 "pole_position": ["Max Verstappen"],
 "podium": ["Max Verstappen",
   "Lewis Hamilton", "Carlos Sainz"],
 "starters": 19, "finishers": 14,
 "retirements": ["Sergio Perez: Engine",
   "Nicholas Latifi: Accident", ...],
 "driver_ids": ["driver_max_verstappen",
   "driver_hamilton", ...],
 "team_ids": ["team_red_bull",
   "team_mercedes", ...],
 "summary": "The 2021 Abu Dhabi Grand
   Prix was a Formula One motor race...",
 "race": "The race started at 17:00
   local time on Sunday...",
 "post_race": "In the laps before the
   safety car, Hamilton was leading...",
 "word_count": 5927,
 "wikipedia_revid": 1373349274
}
```

B Relevance Criteria and Queries

Relevance criteria and query of each information need. The query `q` is the same in all three setups and is parsed by eDisMax with the default operator `OR`, so a document does not need to contain every word. The boosted setup also applies the filter `fq=type:<type>` and the field weights of Table 9; the baseline searches every document in the `all_text` field, and the schemaless setup searches the title and the ten text sections of an index without our schema. Results are sorted by score, and the top 10 are judged.

ID	Type	Relevant	Not relevant	Query q
N1	race	The text says the pit-stop or tyre strategy (number or timing of stops, tyre choice, an undercut) was decisive for who won.	Routine pit stops only, or a strategy that changed positions outside the fight for the victory.	`"pit stop" strategy undercut tyres "two-stop" "one-stop" gamble`
N2	race	Rain or wet conditions are mentioned **and** the result is described as unusual (first win, small team, very few finishers).	A wet race won by the favorite, or a surprising result unrelated to the weather.	`rain wet surprise shock unexpected "first win" "only three"`
N3	race	The safety car clearly affected the final order or the fight for the victory, or its use was controversial.	The safety car is only mentioned in passing, with no effect on the result.	`"safety car" "under the safety car"~2 controversial "lost the lead"~3 "wiped out"~2 "lead was"~2 advantage benefited "pitted under"~3 "cost him"~2`

ID	Type	Relevant	Not relevant	Query q
N4	race	A collision or contact between two drivers of the same team during the race.	Collisions between drivers of different teams; near misses.	`"collided with his teammate"~4 "collided with his team-mate"~4 "contact with his teammate"~4 "contact with his team-mate"~4 "teammates collided"~3 "team-mates collided"~3 "teammate collided"~3 "team-mate collided"~3 "the two cars collided"~2`
N5	race	A death during the Grand Prix weekend (practice, qualifying or race).	Deaths at other events mentioned only as context; serious accidents without fatalities.	`fatal accident killed died death`
N6	race	A disqualification or exclusion from the race or qualifying for a **technical** reason.	Disqualifications for behavior (ignoring a black flag, pushing the car) or time penalties.	`disqualified disqualification illegal underweight technical infringement excluded`
N7	driver	The biography describes a serious injury that interrupted the career **and** a return to racing.	Injuries without a break in the career; injuries that ended it; fatal accidents.	`serious injury injured recovered returned comeback hospital burns`
N8	driver	The biography mentions a win or a notable result at Le Mans or the Indianapolis 500.	Participation without a notable result; Le Mans mentioned for other reasons, such as the 1955 disaster.	`"Le Mans" Indianapolis "Indy 500" won victory`
N9	team	The team history describes a withdrawal or collapse caused by financial difficulties.	Teams sold or renamed without a financial crisis; financial problems the team overcame.	`financial bankrupt bankruptcy debts administration collapsed funding liquidation`
N10	team	The team created or pioneered a technical solution that was later banned.	General bans that affected every team equally, such as the end of turbo engines in 1989; innovations never banned.	`innovation banned outlawed prohibited "fan car" "six-wheeled" "ground effect"`
N11	team	The team changed its name after being sold, or was acquired by another company or owner.	Changes of sponsor or engine only; teams that simply closed.	`renamed rebranded "taken over" takeover bought purchased acquired sold`
N12	circuit	The text links a change to the layout (a chicane, a redesigned corner, a shorter track) to a serious accident.	Accidents without a layout change; changes for commercial or length reasons.	`accident crash fatal death chicane redesigned modified safety layout changed altered`
N13	circuit	Drivers or teams boycotted or refused to race at the circuit for safety reasons, or considered it too dangerous.	Political or contractual boycotts; accidents mentioned without criticism of the circuit's safety.	`boycott boycotted refused dangerous unsafe "safety concerns" protest criticized drivers`

C Additional Figures

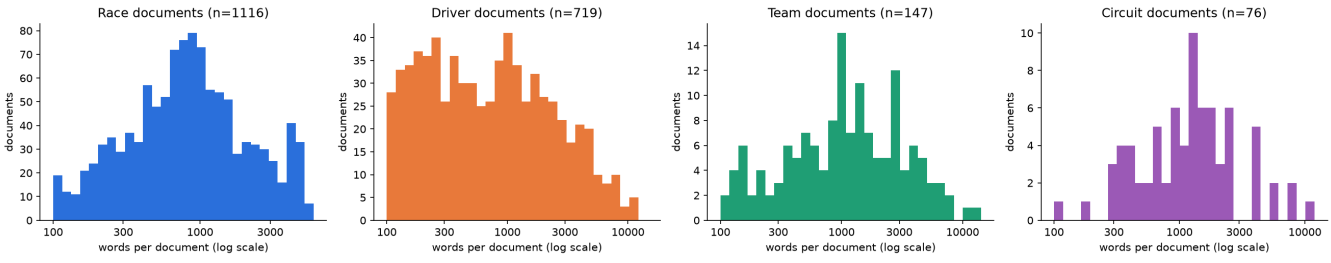


Figure 7: Words per document, by document type (log scale)

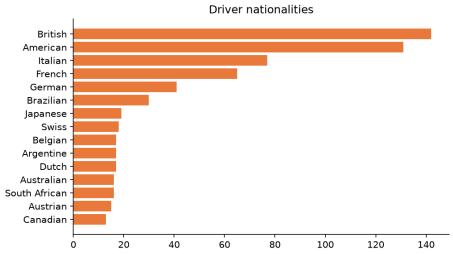


Figure 8: Driver nationalities

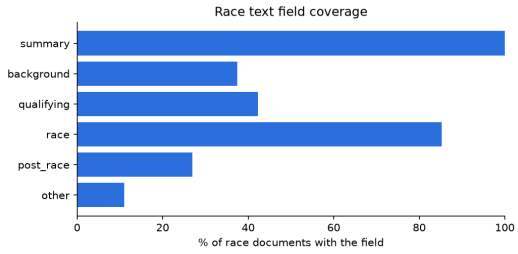


Figure 9: Share of race documents with each text field

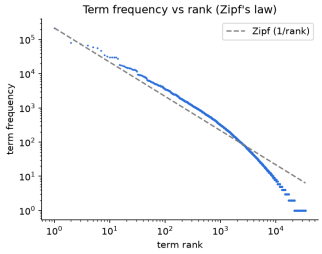


Figure 10: Term frequency versus rank

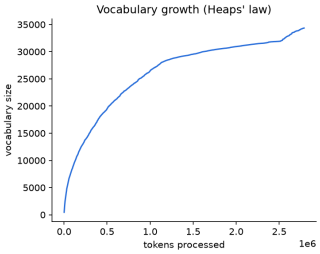


Figure 11: Vocabulary growth

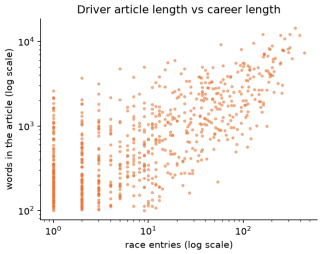


Figure 12: Driver article length versus race entries

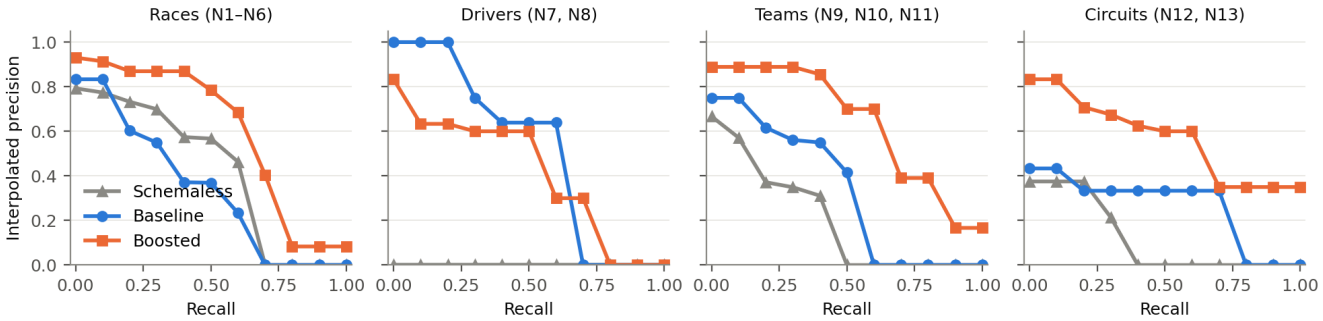


Figure 13: Interpolated precision-recall curve of each setup, by document type

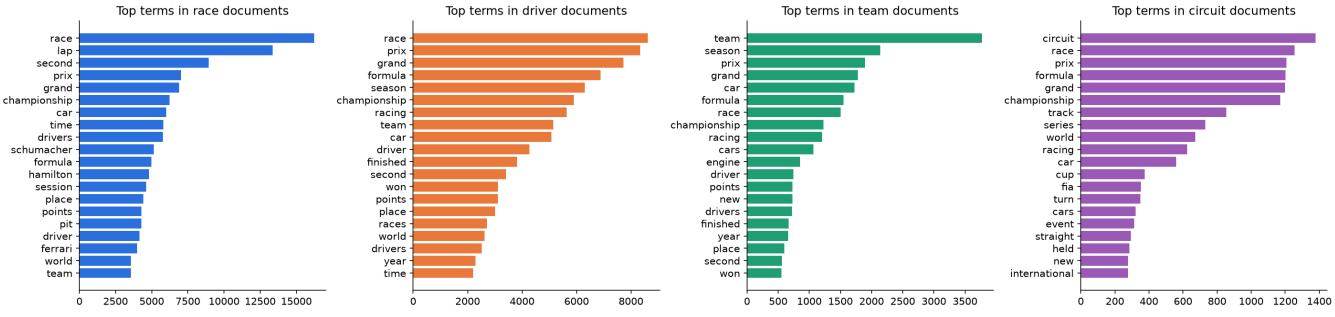


Figure 14: Most frequent terms per document type, without stopwords

D Domain Conceptual Model

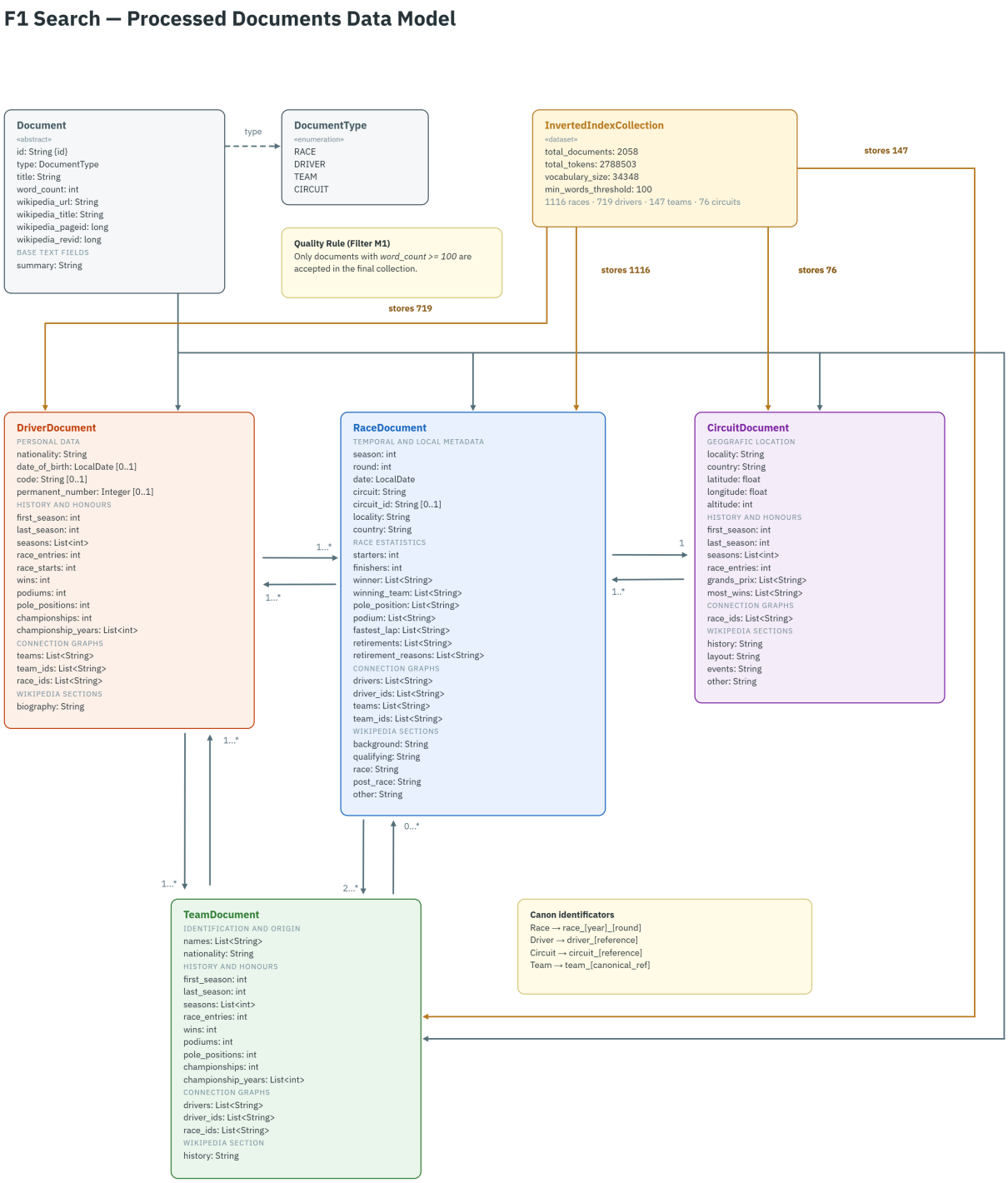


Figure 15: Domain conceptual model.